DEVELOPMENT STORY

AI 에이전트 팀으로 차세대 추천 시스템을 만들기까지

데스크톱 GPU 1대, 3인 팀, 그리고 AI 협업의 기록

"AI가 코드를 쓰지만,
설계 판단은 사람이 한다."

인프라 예산 없이, 데스크톱 GPU 1대와 AI 에이전트들의 조합으로 14-task, 7-expert PLE+adaTT 추천 시스템을 구축한 과정을 기록한다.

AIOps PLE Platform

프로젝트 배경

팀 구성과 제약조건

프로젝트 팀은 3명이었다. 데이터사이언티스트 겸 PM/리드 아키텍트 1명과 엔지니어 2명. 전용 인프라 예산은 없었고, 개발에 사용할 수 있는 GPU는 로컬 PC에 장착된 데스크톱용 RTX 4070 (12GB VRAM) 1대가 전부였다.

> **제약조건 요약**
>
> - **팀 규모**: 3명 (PM/리드 아키텍트/데이터사이언티스트 1 + 엔지니어 2)
> - **인프라**: 전용 GPU 서버 없음. RTX 4070 12GB 1대 (로컬)
> - **예산**: 인프라 구매 예산 없음. AWS SageMaker spot 인스턴스 활용
> - **일정**: 기존 ALS 기반 추천 시스템을 대체할 차세대 시스템 구축

인프라 제약의 현실

조직으로부터 받은 지원은 사실상 전무했다. GPU가 필요하다고 요청해도 "어쩔 수 없다"는 답변뿐이었고, AI 도구 구독료(Claude Code, Gemini, Cursor), AWS 클라우드 비용(SageMaker Spot 인스턴스, S3 스토리지), 부속기기, 식비 등 프로젝트에 소요된 모든 비용은 PM/리드 아키텍트 개인 자금으로 충당했다.

데이터 수집 환경도 열악했다. Spark나 Impala로의 전환 요청이 거부되어 HIVE 기반의 병목 환경에서 작업해야 했고, 이를 극복하기 위해 병렬 쿼리 로직을 직접 설계하여 네트워크 대역폭을 최대한 활용했다.

작업 공간은 열순환이 되지 않는 서버실 옆 공간으로, 적절한 냉방도 제공되지 않았다.

함께 일한 팀원 2명은 정식 계약이 아닌 청년 자문위원 자격의 서포터즈로, 대학 졸업 후 취업을 준비하는 과정에서 프로젝트에 참여했다.

이 모든 제약이 오히려 설계의 개성을 발현시켰다.

생물의 진화에서 선택압(selection pressure)이 종의 특화를 이끌듯, 자원의 제약은 설계에 선택압으로 작용했다. 12GB VRAM이라는 한계는 "파라미터를 늘려서 성능을 올리자"는 안이한 접근을 원천적으로 차단했고, 대신 "구조적 귀납 편향으로 표현력을 확보하자"는 방향으로 진화를 강제했다. 이종 전문가 7개가 각자 다른 수학적 관점을 인코딩하는 설계, 11개 학문 분야에서 구조적 동형사상을 빌려온 피처 엔지니어링, FP32에서도 돌아가도록 각 전문가를 경량화한 선택 — 이 모든 것이 제약 속에서 나온 적응의 결과물이다.

대형 GPU 클러스터가 있었다면 이 아키텍처는 탄생하지 않았을 것이다. Transformer 기반 대형 전문가를 7개 쌓고 파라미터로 밀어붙이는 평범한 접근을 택했을 가능성이 높다. 제약이 없었다면 개성도 없었다.

온프렘 시스템 규모

온프렘 시스템은 단순한 프로토타입이 아니라 프로덕션 규모의 시스템이었다. 80개 이상의 Airflow DAG, Champion-Challenger 모델 경쟁, 주간 자동 재학습, 734D 피처 텐서, 16개 동시 태스크, 62개 데이터 테이블 인제스천을 포함한다. 이 규모의 시스템을 3명이 구축한 것 자체가 AI 에이전트 활용의 결과물이다.

프로젝트 목표

기존 금융 상품 추천 시스템은 ALS (Alternating Least Squares) 기반의 협업 필터링이었다. 이를 PLE (Progressive Layered Extraction) + adaTT (Adaptive Task Transfer) 아키텍처 기반의 멀티태스크 딥러닝 추천 시스템으로 교체하는 것이 목표였다. 14개 태스크를 7개 전문가 네트워크가 처리하며, 태스크 간 관계를 명시적으로 모델링하는 구조다.

아키텍처 의사결정 여정

최종 아키텍처에 도달하기까지 여러 후보를 탐색하고 기각하는 과정을 거쳤다.

기존 시스템은 ALS 기반 협업 필터링이었다. 첫 번째 대안으로 **Black-Litterman 모델**을 검토했다. 포트폴리오 이론에서 온 이 모델은 전문가 의견(뷰)과 시장 균형을 베이지안 업데이트로 결합한다. 그러나 사후분포로 혼합된 결과에서 "어떤 모델이 얼마나 기여했는가"를 분리할 수 없었다. 금융 현장에서는 고객, 영업점, 규제기관 모두에게 추천 근거를 설명해야 하는데, 베이지안 업데이트가 개별 모델의 기여를 불투명하게 만들어 비즈니스 설명이 불가능했다.

두 번째 대안은 **모델 앙상블**이었다. N개 모델을 독립적으로 학습시키고 결과를 결합하는 방식이다. 그러나 N개의 관리 포인트, N배의 서빙 비용이 발생하고, "MLP 3번이 28% 기여했다"는 식의 설명은 비즈니스적으로 무의미했다. 비용과 설명 가능성 모두에서 문제가 있었다.

이 과정에서 핵심적인 리프레이밍이 발생했다: 전문가들을 모델 **밖에서** 결합하는 것이 아니라, 단일 모델 **안에서** 결합하면 어떨까? 이것이 PLE의 이종 전문가 아키텍처로의 전환 계기였다.

태스크 그룹 설계에서도 시행착오가 있었다. 초기에는 **GMM 클러스터 서브헤드** 방식을 시도했다 — K개 클러스터 $\times$ T개 태스크 = $K \times T$ 복잡도 폭발이 발생했다. 이를 4개의 Financial DNA 태스크 그룹(상품 보유 확률, 다음 상품, 고객 가치, 이탈 위험)으로 전환하여 구조를 안정화했다.

AI 에이전트 조직화

Phase별 AI 도구 운용

프로젝트의 각 단계에서 서로 다른 AI 도구를 전략적으로 배치했다. 각 도구의 강점을 살려 역할을 분담하는 것이 핵심이었다.

Phase A: 아이디에이션 (Gemini)

초기 컨셉 탐색과 브레인스토밍에는 Gemini를 활용했다. 광범위한 지식 베이스를 바탕으로 아키텍처 후보군을 빠르게 스캔하고, 다양한 접근법의 장단점을 비교하는 데 효과적이었다. ALS 대체 옵션, Black-Litterman 탐색, 모델 앙상블 접근법 등 다양한 아키텍처 후보를 Gemini와의 대화를 통해 탐색했다.

가장 큰 가치를 발휘한 것은 학제간 피처 아이디어의 탐색이었다. "화학 반응속도론으로 소비 행동을 설명할 수 있는가?", "상품 채택이 전염병 확산과 구조적으로 동등한가?" 같은 질문을 던지며, 금융 고객 행동과 구조적으로 동형인 문제를 가진 학문 분야를 체계적으로 식별했다. PM/리드 아키텍트가 도메인 전문성(FRM, 신용 분석 경력)을 제공하고, Gemini가 학제간 연결고리를 제안하는 방식으로 협업이 이루어졌다. 이 과정에서 "구조적 동형사상"이라는 핵심 개념이 자연스럽게 부상했다.

Gemini의 광범위한 지식은 특정 기술의 깊이보다는 "어떤 분야에서 비슷한 문제를 이미 풀었는가"를 빠르게 탐색하는 데 최적이었다. 11개 학문 분야에서 피처를 도입하겠다는 설계 방향이 이 단계에서 확립되었고, 이후 모든 기술적 결정의 근간이 되었다.

Phase B: 기술 검증 (Claude Opus)

아이디에이션 결과를 구체적인 아키텍처로 발전시키는 단계에서는 Claude Opus를 사용했다. 수학적 엄밀성이 필요한 loss function 설계, 데이터 리키지 검증, 정규화 파이프라인 설계 등 기술적 깊이가 요구되는 작업에 집중했다.

각 전문가의 실현 가능성을 하나씩 검증했다: "HGCN이 MCC 계층 구조에서 작동하는가?", "Mamba가 17개월 시퀀스에 충분히 효율적인가?" 같은 질문에 대해 Opus와 심층적인 기술 토론을 진행했다. PLE vs MMoE의 트레이드오프 분석, adaTT의 loss-level vs representation-level 전이 비교 등 아키텍처 수준의 깊은 분석이 이 단계에서 이루어졌다. 특히 동질적 MoE에서 발생하는 전문가 붕괴(expert collapse) 문제를 Opus와의 대화에서 먼저 식별한 후, NeurIPS 2024의 sigmoid gate 논문을 발견하는 계기가 되었다.

Opus는 가정을 도전하는 데도 효과적이었다. "Black-Litterman이 정말 적합한가?"라는 반론을 제기하여 PLE로의 전환을 촉진했다. 이 단계에서 19개의 기술 참조 문서(.typ 파일)가 Opus와의 공동 작업으로 작성되었으며, 이 문서들은 이후 구현 단계에서 각 에이전트가 참조하는 설계 명세서 역할을 했다.

Phase C: 코드 환경 정비 (Cursor)

GitHub 기반의 코드 환경 구성, 프로젝트 구조 설계, 초기 보일러플레이트 생성은 Cursor로 수행했다. IDE 통합 환경에서의 빠른 코드 네비게이션과 리팩토링이 강점이었다.

이 단계에서 6개의 초기 설계 문서(00-09 아키텍처 명세)가 작성되었고, 가장 중요한 산출물은 CLAUDE.md 가드레일의 수립이었다. Config-driven 원칙, 관심사 분리, 리키지 방지 규칙 등 이후 모든 AI 에이전트가 따라야 할 "헌법"이 코드 한 줄 작성되기 전에 확립되었다. 이것은 의도적인 순서였다 — 가드레일이 먼저이고, 에이전트 실행이 그 다음이다.

Phase D: 병렬 구현 (Claude Code — Opus/Sonnet)

본격적인 구현 단계에서는 각 팀원이 AI 에이전트의 "팀장" 역할을 맡았다. Claude Code 환경에서 Opus와 Sonnet을 병렬로 운용하여 서로 다른 모듈을 동시에 구현했다. 이 단계가 가장 집약적이었으며, 3명의 인간이 각각 AI 에이전트 팀을 이끌었다.

PM/리드 아키텍트의 AI 팀은 아키텍처 수준의 결정(PLE config, adaTT 태스크 그룹, 로짓 전이 설계)에는 Opus를, 빠른 코드 구현(generator, adapter, pipeline runner)에는 Sonnet을 배치했다. 이 과정에서 label leakage 3건 탐지, FP16 NaN 원인 4가지 진단, GPU 활용률 최적화 등 핵심적인 디버깅 세션이 이루어졌다. 엔지니어 1의 AI 팀은 데이터 수집 파이프라인, HIVE 병렬 쿼리 로직, 10개 generator(TDA, HMM, Mamba, Graph, GMM 등)의 피처 엔지니어링, 피처-비즈니스 역매핑 레지스트리를 담당했다. 엔지니어 2의 AI 팀은 모델 학습, 수학적 검증, 지식 증류(PLE에서 LGBM으로)를 담당했다.

각 팀이 병렬로 작업하면서도 일관성을 유지할 수 있었던 것은 Phase C에서 수립한 CLAUDE.md 가드레일과 인터페이스 계약 검증 프로세스 덕분이었다. 파일 A가 저장하는 키와 파일 B가 읽는 키가 일치하는지를 매 병렬 작업 후 반드시 검증했고, 이를 통해 통합 시 발생할 수 있는 인터페이스 불일치를 사전에 방지했다.

Phase E: 실험 + 논문 (Claude Code Extension)

ablation 실험 단계에서는 Claude Code를 실시간 모니터링 도구로 활용했다. ablation 진행 상황, GPU 활용률, 에러 탐지를 실시간으로 수행했다. 이 과정에서 PLE toggle 버그를 라이브 디버깅으로 발견했다 — `use_ple=false` 설정이 전문가 구성 자체를 변경하여 공정한 비교가 불가능했던 문제였다.

실험 대기 시간을 활용한 문헌 조사도 이루어졌다. PLE의 val_loss가 수렴하지 않는 현상을 관찰하던 중, Opus와의 대화를 통해 softmax gate의 경쟁적 특성이 이종 전문가 간 수렴을 방해한다는 가설을 세웠고, NeurIPS 2024의 sigmoid gate 논문을 찾아 이론적 근거를 확보했다. 실험 결과 분석과 논문 작성이 동시에 진행되는 방식이었다.

논문 작성 단계에서는 4편의 논문(영문/한국어), 22개의 기술 문서가 Claude와의 반복적 작업으로 생성되고 다듬어졌다. 개발 스토리 자체도 Claude와 함께 프로젝트 과정을 되돌아보며 작성되었다. 이 단계에서 AI는 단순한 텍스트 생성기가 아니라, 프로젝트의 의미를 함께 구성하는 사고 파트너 역할을 했다.

문서 생산 규모

AI 에이전트와의 협업은 코드 구현에 그치지 않았다. 온프렘 프로젝트에서만 260개 이상의 문서(설계서 28개, 기술참조서 19개, 코드 리뷰 16건, 보고서 95건, 가이드 5개)가 생산되었으며, 총 30MB 이상의 기술 문서가 작성되었다. 이 중 상당수는 AI와 공동 작성 또는 AI가 초안을 작성하고 인간이 검수하는 방식으로 제작되었다. 특히 "소네트 작업 검증 리포트"는 Opus가 Sonnet의 코드를 검증하는 AI 간 리뷰 프로세스를 보여주며, "Claude Code Opus용 500+ 항목 체크리스트"는 AI 에이전트에게 체계적 검수 업무를 위임하는 방법론을 보여준다.

메모리 뱅크와 가드레일 시스템

온프렘 프로젝트에서 확립된 AI 관리 체계는 AWS 프로젝트에도 그대로 이식되었다. memory-bank 시스템(8개 컨텍스트 파일: projectbrief, activeContext, progress, techContext, productContext, systemPatterns, tasks, style-guide)으로 세션 간 맥락을 유지하고, .claude/RULES.md로 코딩 규칙을 강제하며, .cursorrules와 동기화하여 Cursor AI와 Claude Code가 동일한 가드레일을 따르도록 했다. 심지어 Claude, Codex, Vertex AI 세 플랫폼에서 자동화 실험 브랜치(exp/claude-auto-*, exp/codex-auto-*, exp/vertex-auto-*)를 운영하여 AI 도구 간 비교 실험도 수행했다.

왜 Claude Code여야 했는가

이 프로젝트의 복잡도에서 결정적이었던 것은 긴 맥락 유지(1M context), 세션 간 메모리 뱅크, 서브에이전트 병렬 실행이었다.

Label leakage 3건의 연쇄 추적은 며칠에 걸친 작업의 맥락을 유지해야만 가능했다. 첫 번째 leakage(has_nba 중복 컬럼)를 수정한 후, 같은 세션에서 두 번째(ground truth glob 정렬), 세 번째(generator label 입력)를 발견할 수 있었던 것은 이전 수정의 맥락이 살아있었기 때문이다.

FP16 NaN 4개의 동시 진단(CGC entropy, OT Sinkhorn, Causal DAG, logits)은 모델 아키텍처 전체를 한 번에 조망하면서 각 expert의 수치 연산을 추적해야 했다. 이는 파일 하나씩 보는 방식으로는 불가능했다.

실험 대기 중 NeurIPS 2024 sigmoid 논문을 발견하고, 우리 실험의 PLE softmax 미수렴 관찰과 연결하여 sigmoid gate를 구현한 과정도 연속된 맥락 안에서 이루어졌다 — 실험 결과 분석 → 문헌 탐색 → 이론 연결 → 코드 구현이 하나의 흐름으로 진행되었다.

논문 4편과 기술 문서 22편의 일관성을 유지하면서 동시 수정하는 것도, 전체 문서 체계를 기억하고 있는 에이전트만이 가능한 작업이었다.

AI 협업에서 발견된 핵심 패턴

프로젝트 전 과정에서 AI 협업의 반복적 패턴이 드러났다. 이 패턴들은 의도적으로 설계된 것이 아니라, 실제 작업 과정에서 자연스럽게 발현된 것이다.

> **1. "AI는 HOW, 인간은 WHAT과 WHY를 결정한다"**

: AI가 코드와 텍스트를 생성하지만, 아키텍처 결정은 인간이 내린다. 구조적 동형사상이라는 핵심 통찰은 인간-AI 대화에서 발현되었지만, 그것을 설계 원칙으로 채택한 것은 인간의 판단이었다.

2. "에이전트 전에 가드레일을 세운다"

: CLAUDE.md는 코드가 아니라 코드 이전에 작성되었다. 헌법이 먼저이고 입법이 나중인 것처럼, 가드레일이 먼저이고 에이전트 실행이 나중이다.

3. "이종 AI = 이종 전문가"

: 모델의 이종 전문가 설계가 개발 도구 선택에도 그대로 적용되었다. 각 AI 도구가 특화된 역할을 수행하면서, 단일 도구로는 달성할 수 없는 품질과 속도를 확보했다.

4. "메모리 뱅크로 연속성 확보"

: 세션 간 맥락 보존을 위한 영속 파일 시스템이 AI 에이전트의 가장 큰 약점인 "맥락 망실"을 극복하는 핵심 메커니즘이었다.

5. "AI로 빠르게 실패한다"

: leakage, FP16 NaN, ablation 필터 미작동 등 수동으로는 수일이 걸릴 버그를 AI 에이전트와 함께 수분 내에 탐지하고 수정했다. 빠른 실패가 빠른 학습으로 이어졌다.

6. "AI는 코드 머신이 아니라 사고 파트너"

: 구조적 동형사상이라는 통찰, sigmoid gate 가설, 전문가 붕괴 문제의 식별 등 프로젝트의 핵심적인 지적 돌파구가 인간-AI 대화에서 나왔다.

Phase	AI 도구	역할
A. 아이디에이션	Gemini	컨셉 탐색, 아키텍처 후보 스캔, 학제간 브레인스토밍
B. 기술 검증	Claude Opus	수학적 검증, loss 설계, 리키지 분석, 기술 문서 19편 공동 작성
C. 환경 정비	Cursor	GitHub 구조, CLAUDE.md 가드레일, 설계 문서 6편
D. 병렬 구현	Claude Code (Opus/Sonnet)	3인 x AI 팀 병렬 코딩, 디버깅, 10개 generator 구현
E. 실험 + 논문	Claude Code Extension	실시간 모니터링, 문헌 조사, 논문 4편 + 기술 문서 22편

품질 관리 전략

CLAUDE.md 가드레일

AI 에이전트가 생성하는 코드의 품질을 보장하기 위해, 프로젝트 루트에 CLAUDE.md 파일을 두어 가드레일을 설정했다. 이 파일은 AI 에이전트가 매 세션 시작 시 읽어들이는 시스템 지침이다.

> **CLAUDE.md 핵심 가드레일**
>
> - **Config-Driven 원칙**: 모든 파라미터는 YAML에서 읽는다. 하드코딩 금지.
> - **관심사 분리**: Adapter는 데이터 변환만, Runner는 파이프라인만, train.py는 학습만.
> - **데이터 리키지 방지**: Scaler는 train split에서만 fit. temporal split에 gap_days 필수.
> - **코드 검수 4단계**: 컴파일 검증 → 인터페이스 계약 검증 → 하드코딩 스캔 → 관심사 분리 검증.

메모리 뱅크

AI 에이전트는 대화 세션이 끝나면 맥락을 잃는다. 이 문제를 해결하기 위해 "메모리 뱅크" 시스템을 도입했다. 세션 진행 상황, 설계 결정 사유, 피드백 이력을 구조화된 마크다운 파일로 관리하여, 새 세션이 시작될 때 AI가 이전 맥락을 빠르게 복원할 수 있게 했다.

인터페이스 계약 검증

병렬 에이전트가 서로 다른 모듈을 동시에 수정할 때 가장 큰 위험은 인터페이스 불일치다. 파일 A가 저장하는 키 이름과 파일 B가 읽는 키 이름이 달라지는 문제를 방지하기 위해, 병렬 작업 후에는 반드시 "인터페이스 계약 검증" 단계를 수행했다. cross-file 키/필드 매핑 테이블을 자동 생성하여 불일치를 사전에 탐지했다.

기술적 도전과 해결

개발 과정에서 마주친 20건 이상의 기술적 문제를 5개 범주로 정리한다. 단순한 디버깅 기록이 아니라, 각 범주가 데스크톱 GPU 환경에서의 대규모 멀티태스크 학습이 요구하는 엔지니어링 역량의 단면을 보여준다.

데이터 무결성 (Data Integrity)

학습 데이터의 오염은 모델 성능을 무의미하게 만든다. 이 프로젝트에서는 레이블 리키지와 스키마 불일치가 반복적으로 발생했으며, 각각을 체계적으로 탐지하고 방어 체계를 구축했다.

Label Leakage 3건

모델 학습 초기에 비정상적으로 높은 성능(AUC=1.0)이 관측되어 3건의 리키지를 발견했다.

> - **중복 컬럼**: `has_nba_1`이 레이블과 상관계수 1.0으로 존재. EXCLUDE 처리로 해결.
> - **파일 로딩 순서**: glob 알파벳 정렬로 `ground_truth.parquet`가 `benchmark.parquet`보다 먼저 로드됨. 하위 디렉토리 분리로 해결.
> - **Generator 입력 오염**: GMM 등의 generator가 레이블 컬럼을 입력으로 사용. `label_cols` 자동 제외 로직 추가.

LeakageValidator를 학습 전 필수 단계로 추가하고, CLAUDE.md 가드레일에 검증 규칙을 명시하여 재발을 방지했다.

apply_ablation Schema 미갱신

Ablation 필터가 텐서에서 피처를 제거하는 데 성공했지만, `feature_schema["columns"]`가 원래의 316개 컬럼을 유지하여 모델이 매번 동일한 316차원 입력을 받았다. `apply_ablation`에서 텐서 제거와 동시에 `columns`, `num_features`, `feature_group_ranges`를 함께 갱신하도록 수정하여 해결했다.

FeatureRouter 활성화 — Expert별 피처 서브셋 라우팅

마일스톤: FeatureRouter가 활성화되어 각 expert가 전체 316D 피처 중 자신에게 지정된 feature group만 입력으로 받게 됐다. Expert별 입력 차원: deepfm=109D, temporal_ensemble=129D, hgcn=34D, perslay=32D, causal=103D, lightgcn=66D, optimal_transport=69D. 모델 파라미터가 4.77M → 2.8M으로 감소했다 (feature_group_ranges 패치 후 최종 기준).

구현 과정에서 두 가지 버그가 발생했다:

- **Config scoping 오류**: FeatureRouter가 feature_group_ranges를 읽을 때 잘못된 config 레벨을 참조하여 라우팅이 전혀 작동하지 않는 문제. config 경로를 명시적으로 지정하여 해결했다.
- **`shared_{i}` 이름 매핑 오류**: CGCLayer가 shared expert를 `shared_0`, `shared_1` 형태로 등록하지만, FeatureRouter가 이를 feature group key와 매핑할 때 인덱스 불일치가 발생. expert 이름 매핑 테이블을 별도로 관리하도록 수정했다.

`feature_groups.yaml`의 `target_experts` 선언이 실제 런타임 라우팅을 결정하며, 코드에는 expert 이름이나 컬럼명이 하드코딩되지 않는다.

adaTT 포팅 버그 5건 — 온프렘 코드 비교로 발견

배경: Structure ablation에서 adaTT가 일관적으로 성능을 하락시켰다 (sigmoid: −0.006, softmax: −0.021, no PLE: −0.004). adaTT의 설계 문제가 아니라 포팅 과정의 구현 오류였다.

원인 발견: 온프렘(gotothemoon) 소스코드와 1:1 비교하여 5건의 버그를 발견했다.

1. **Gradient 추출 빈도**: AWS는 에포크 마지막 배치에서만 gradient를 추출 (1회/epoch). 온프렘은 10스텝마다 추출 (17회/epoch). `_is_epoch_end_step` 플래그 대신 `global_step % grad_interval`을 사용하도록 수정.
2. **Config 로드 경로**: pipeline.yaml의 root-level `adatt:` 섹션이 `model_config`이나 `label_schema`에서 읽히지 않았다. `config.get("adatt")` fallback을 추가.
3. **freeze_epoch 미전달**: `AdaTTConfig` 생성 시 `freeze_epoch`을 전달하지 않아 항상 None. transfer weight가 끝까지 불안정하게 적응.
4. **Loss 구조**: 온프렘은 uncertainty weighting을 먼저 적용 (loss scale 정규화) 후 adaTT transfer. AWS는 either/or로 구현되어 adaTT 활성화 시 uncertainty weighting이 꺼졌다. 14개 태스크의 loss scale이 제각각인 상태에서 transfer → 큰 loss가 지배.
5. **warmup_epochs: 0**: affinity matrix가 identity (측정 없음) 상태에서 즉시 transfer 시작. 의미 없는 loss 공유.

수정 결과: sigmoid_adatt AUC 0.5605 → 0.5746 (+0.014). 피크(Ep6)에서 0.5786으로 sigmoid baseline(0.5771)을 초과.

교훈: preflight 로그 (`"AdaTT config: warmup=X, freeze=X, source=X"`)를 추가하여 config 적용 여부를 학습 시작 전에 검증하도록 했다. MLflow가 있었다면 이 삽질을 상당 부분 방지할 수 있었을 것이다.

수치 안정성 (Numerical Stability)

Mixed precision 학습은 속도를 2배 높이지만, FP16/BFloat16의 좁은 표현 범위가 NaN 전파를 유발한다. 4건의 underflow와 2건의 변환 오류가 발생했다.

FP16 Underflow와 NaN 전파

Phase 2 AMP 학습 중 CGC 엔트로피, OT Sinkhorn, Causal DAG, logits 연산에서 FP16 underflow로 NaN이 전파되었다. 작은 gradient 값들이 FP16 범위 밖으로 떨어진 것이 원인이었다.

BFloat16 NumPy 변환 및 GradScaler

BFloat16 텐서를 `.numpy()`로 변환 시 NumPy가 해당 dtype을 지원하지 않아 모든 validation metrics 계산이 실패했다. 모든 텐서에 `.float()` 캐스트 후 `.numpy()` 호출 패턴을 적용했다. 또한 모든 배치가 NaN인 극단적 상황에서 `scaler.step()`이 "No inf checks were recorded" assertion을 발생시켰다. backward count가 0이면 step을 건너뛰는 방어 로직을 추가했다.

인프라 및 환경 (Infrastructure)

데스크톱 GPU 1대 환경에서는 드라이버 충돌, 백그라운드 프로세스, 네트워크 제약이 실험 설계 자체를 바꿀 수 있다.

Docker GPU Passthrough 및 좀비 컨테이너

Windows에서 Docker GPU passthrough가 CUDA 버전 불일치로 불안정하여 로컬 Python 환경으로 전환했다. 또한 종료되지 않은 좀비 컨테이너가 GPU 메모리를 점유하여 학습 속도가 1/3로 저하되는 문제도 발생했다.

torch CPU/CUDA 버전 충돌

SageMaker SDK v3(3.7.0) 설치 시 torch가 CPU 버전으로 교체되어 GPU 학습이 불가능해졌다. SageMaker v2(2.257.1)로 고정하여 해결했다. conda 환경에서도 CPU/CUDA 빌드 충돌이 반복되어 캐시 정리 후 CUDA 버전을 명시적으로 지정하여 재구성했다.

torch conda 캐시 복구 및 Ollama GPU 점유

관공서 네트워크에서 `download.pytorch.org`가 방화벽 차단(403)되어 conda 캐시의 기존 패키지를 수동 복사하여 환경을 복구했다. 별도로, Ollama 자동 시작이 VRAM 2GB를 점유하여 batch size 선택에 영향을 미쳤다. 12GB VRAM 환경에서 백그라운드 프로세스 하나가 실험 설계를 바꿀 수 있다.

VRAM Spillover 분석

batch 6144에서 전용 12GB + 공유 11GB = 23GB(시나리오당 10시간), batch 2048에서 전용 9GB + 공유 0.1GB = 9.1GB(시나리오당 2시간). 공유 GPU 메모리는 PCIe 경유로 전용 VRAM 대비 10–20배 느렸다. 이 정량 분석으로 최적 batch size를 결정했다.

파이프라인 엔지니어링 (Pipeline Engineering)

대규모 데이터 처리와 ablation 오케스트레이션에서 발생한 시스템 수준의 문제들이다.

pandas에서 DuckDB/cuDF 전환

941K 행의 pandas 처리에서 메모리가 급증했다. DuckDB(CPU columnar)와 cuDF(GPU)로 전환하여 메모리 효율성과 처리 속도를 동시에 개선했다.

NVIDIA Merlin 생태계 평가와 선택적 채택

초기에 NVIDIA Merlin 생태계(NVTabular, HugeCTR 등)를 풀스택 솔루션으로 도입하려 했다. 그러나 본 시스템의 7종 이종 전문가 구조(DeepFM, Temporal Ensemble, HGCN, PersLay, LightGCN, Causal, OT)가 Merlin의 정형화된 파이프라인과 맞지 않았다. Merlin은 단일 모델 학습에 최적화되어 있어, 전문가별로 서로 다른 입력 형식과 연산 그래프를 요구하는 이종 아키텍처를 수용하기 어려웠다. 최종적으로 Merlin에서는 DataLoader 컴포넌트만 채택하

고, 데이터 전처리와 피처 엔지니어링에는 cuDF를 직접 활용하며, 서빙/배포에는 Triton Inference Server를 도입했다. 풀스택 프레임워크를 평가하되 실제로 맞는 컴포넌트만 취하는 실용적 엔지니어링 철학을 반영한 결정이었다.

Subprocess Pipe Deadlock

ablation 오케스트레이터가 `subprocess.run(capture_output=True)`로 시나리오를 실행했으나, 대량의 stdout이 파이프 버퍼(64KB)를 초과하여 교착 상태가 발생했다. stdout/stderr를 파일로 리다이렉트하여 해결했다.

Ground Truth 파일 오류 로드

glob 알파벳 정렬로 `benchmark_ground_truth.parquet`가 원본 데이터보다 먼저 로드되어 Phase 0가 정답 변수로 피처를 생성했다. ground truth 파일을 하위 디렉토리로 분리하여 해결했다.

Batch Size 불일치 및 bash JSON 이스케이프

`pipeline.yaml`의 batch_size=2048을 `run_ablation_manual.sh`가 6144로 오버라이드하여 VRAM spillover가 발생했다. 모든 설정을 config 단일 소스로 통일했다. bash 스크립트에서 JSON 파라미터의 이스케이프 처리 실패도 별도로 수정했다.

모델 아키텍처 발견 (Architecture Insights)

ablation 실험과 학습 과정에서 모델 구조에 대한 근본적 발견이 있었다.

PLE Toggle 버그와 Ablation 필터 미작동

`use_ple=false` 설정 시 7개 이종 전문가가 MLP 1개로 축소되어 공정한 비교가 불가능했다. expert basket은 유지하고 PLE layering만 비활성화하도록 수정했다. 또한 `feature_group_ranges`가 컬럼 단위로만 저장되어 ablation 필터의 그룹명 매칭이 실패, 24개 시나리오 전체에서 AUC가 동일(0.913)했다. 그룹 레벨 키를 추가하여 해결했다.

GPU 활용률 최적화

초기에는 batch size 512로 GPU 활용률이 낮았다. batch size 증가, DataLoader 튜닝(num_workers, pin_memory), Phase 0 텐서 사전 저장을 적용하여 학습 처리량을 개선했다. 다만 12GB VRAM 제약으로 batch size는 2048이 상한이었으며, 이를 초과하면 shared GPU memory로 spillover가 발생하여 오히려 성능이 저하되었다.

Softmax vs Sigmoid Gate 발견

PLE의 val_loss가 Phase 2에서 3.702로 고정되고, shared_bottom(1 MLP)이 ple_only(7 expert)보다 낮은 val_loss를 보이는 역전 현상이 관찰되었다. CGC softmax gate의 경쟁적 특성이 이종 전문가 간 수렴을 방해한 것이다. NeurIPS 2024 논문의 sigmoid gate 이론적 우위를 확인하고 구현을 진행했다. 이종 전문가 아키텍처에서 gate 함수 선택이 성능에 결정적 영향을 미친다는 교훈을 얻었다.

설계 철학의 배경

"설득의 대상은 항상 사람이다"

금융 추천 시스템의 최종 소비자는 알고리즘이 아니라 사람이다. 고객은 "왜 이 상품인가?"를 묻고, 영업점 직원은 추천 근거를 설명해야 하며, 규제기관은 모델의 의사결정 과정을 검증한다. 확률값 하나만으로는 이 세 그룹 중 누구도 설득할 수 없다. 따라서 모든 설계 결정의 기준은 "이유를 설명할 수 있는가?"였다.

2축 분해 프레임워크

아키텍처의 핵심 구조는 2축 분해에 기반한다: **Financial DNA**(이 고객은 누구인가?) × **Data Modality**(데이터의 형태는 무엇인가?). Financial DNA 축은 상품 보유 확률, 다음 상품, 고객 가치, 이탈 위험의 4개 태스크 그룹으로 구성된다. Data Modality 축은 상태/스냅샷/시계열/계층/아이템의 5가지 피처 유형에 대응하는 이종 전문가로 구성된다. 이 2축의 교차점이 전체 모델의 학습 구조를 정의한다.

전문가 붕괴 문제와 이종 전문가의 필요성

동질적 MLP 전문가(예: 모두 같은 구조의 3-layer MLP)를 사용하면 학습 과정에서 **전문가 붕괴(expert collapse)**가 발생한다 — 게이팅 네트워크가 하나의 전문가만 선택하고 나머지는 사실상 사용하지 않는 현상이다. Pinterest와 Kuaishou의 대규모 실험에서도 이 문제가 확인되었다. 구조적으로 서로 다른 이종 전문가(LightGCN, Causal OT, TDA, GMM 등)는 입력 공간과 연산 방식이 다르므로 동일한 표현으로 수렴할 수 없어, 붕괴를 구조적으로 방지한다.

피처의 이중 역할: 예측 재료이자 설명 어휘

피처는 예측 성능을 위한 입력일 뿐 아니라, 추천 근거를 설명하는 어휘(vocabulary)이기도 하다. AUC 기여가 미미한 피처라도 영업점에서 "이 고객의 소비 엔트로피가 높아 다양한 상품 경험이 있다"와 같은 설명을 가능하게 한다면, 설명 어휘로서 대체 불가능한 가치를 가진다. 이 때문에 순수한 예측 성능 기준만으로 피처를 제거하지 않았다.

경제학에서 데이터 사이언스까지

이 프로젝트의 설계 철학은 PM/리드 아키텍트의 학문적 여정에서 비롯되었다. 경제학을 전공하며 의사결정 과학(Decision Science)을 배웠고, 금융공학을 거쳐 데이터 사이언스에 이르렀다. 이 과정에서 점점 근본적인 의문이 생겼다: **"데이터 기반 방법론에서 과학은 과연 어디에 있는가?"**

경제학은 수백 년의 과학적 방법론 — 가설 수립, 이론적 프레임워크, 반증 가능성 — 을 축적해왔다. 그러나 경제학 자체도 다른 학문에서 도구를 빌려온 역사가 있다: 일반균형이론은 물리학의 열역학 평형에서, 게임이론은 수학의 조합론에서, 계량경제학은 통계학에서 출발했다. 경제학이 과학인 것이 아니라, **학문 간 도구 전이가 과학 발전의 보편적 패턴**이다. 실제로 경제학 노벨상 수상자 중 상당수가 물리학·수학 출신이다 — Samuelson(열역학→경제균형), Black-Scholes(열전도→옵션가격), Nash(고정점정리→게임이론), Mandelbrot(프랙탈→금융변동성). 경제학의 가장 강력한 도구들이 다른 과학에서 온 것이다.

그러나 금융공학을 거쳐 데이터 사이언스로 오면서, 과학적 엄밀성은 점점 옅어졌다. 머신러닝 모델은 수학적 구조의 납득이 가능했다 — 선형 회귀의 최소자승법, SVM의 마진 최대화, 트리의 정보 이득 등은 왜 작동하는지 설명할 수 있다.

하지만 딥러닝으로 넘어오면서 상황이 달라졌다. "신경망"이라고 하지만 실제 신경계의 구조를 깊이 연구한 것이 아니라 비유적 명칭에 가깝고, "왜 이 가중치가 이 값인가?"에 대한 답은 "데이터가 그렇게 학습시켰다"뿐이다. 설명할 수

있어야 과학이고, 과학철학이라는 학문까지 생겨서 반증가능성, 패러다임 전환 등을 논했는데, 현재의 딥러닝 접근법은 과학이라기보다 엔지니어링에 가깝다고 느꼈다.

구조적 동형사상 — 과학을 다시 가져오기

이 의문에 대한 답이 구조적 동형사상이었다.

이미 인류가 각 학문 분야에서 수백 년에 걸쳐 발견한 과학적 방법론들이 있다. 화학 반응속도론, 역학, 정보이론, 위상수학 — 각각이 주어진 현상에서 어떤 시사점과 인과관계를 끌어낼 수 있는지, 지식의 최전선에서 노력한 결과물이다.

우리가 직면한 문제(금융 고객 행동 이해)의 구조를 제대로 인지했다면, 다른 학문이 이미 풀어놓은 구조적으로 동등한 문제의 해법을 가져올 수 있다. Shannon이 Boltzmann의 열역학 엔트로피를 정보이론에 가져온 것처럼, Black과 Scholes가 열전도 방정식을 옵션 가격에 가져온 것처럼.

이것이 11개 학문 분야의 피처를 도입하고, 각 분야의 수학적 도구에 특화된 이종 전문가를 설계한 근본 동기이다. 단순히 "피처를 많이 만들자"가 아니라, **"어떤 과학적 질문을 던질 것인가"**가 설계의 출발점이었다.

추천 시스템에서 과학적 방법론의 위치

이 프로젝트는 추천 시스템이 단순한 패턴 매칭을 넘어 과학적 이해에 기반할 수 있다는 가능성을 보여주는 시도이다. 모든 추천이 "비슷한 사람들이 이것을 샀다"(상관)에서 끝나는 것이 아니라, "이 고객의 소비 역학이 이러하므로 이 상품이 적합하다"(인과적 설명)로 갈 수 있다.

Pearl의 인과추론, Friedman의 항상소득가설, Boltzmann의 통계역학 — 이들은 각 분야에서 "왜?"에 답한 과학자들이다. 우리의 이종 전문가 아키텍처는 이들의 도구를 금융 추천이라는 맥락에 구조적 동형사상을 통해 가져온 것이며, 이를 통해 추천 시스템에 과학적 설명 가능성을 부여하려는 시도이다.

핵심 교훈

"AI가 코드를 쓰지만, 설계 판단은 사람이 한다"

AI 에이전트는 놀라운 속도로 코드를 생성하지만, 아키텍처 결정, 데이터 리키지 판단, 비용 최적화 전략은 사람의 도메인 지식과 경험에 의존한다. AI는 "어떻게(how)"에 강하지만, "왜(why)"와 "무엇을(what)"은 사람이 정의해야 한다.

> "가장 위험한 순간은 AI가 '그럴듯한 코드'를 만들어낸 직후다.
> 그때 비판적 검토를 멈추면 기술 부채가 쌓인다."

"가드레일 없는 AI 코딩은 기술 부채를 만든다"

CLAUDE.md 없이 AI에게 자유롭게 코딩을 시키면, 하드코딩이 늘어나고 관심사가 뒤섞이며 테스트 불가능한 구조가 만들어진다. 가드레일은 AI의 생산성을 제한하는 것이 아니라, 올바른 방향으로 유도하는 것이다.

"이종 전문가 철학이 개발 방법론에도 적용된다"

PLE 아키텍처의 핵심 철학인 "이종 전문가(Mixture of Experts)"가 개발 방법론 자체에도 적용되었다. Gemini는 넓은 탐색에, Opus는 깊은 분석에, Cursor는 빠른 환경 구성에, Claude Code는 구현에 각각 전문화되었다. 하나의 AI 도구가 모든 것을 하는 것보다, 각 도구의 강점에 맞는 역할을 배분하는 것이 더 효과적이었다.

성과

시스템 구축

추천 시스템

- 14-task 멀티태스크 학습
- 7-expert PLE 네트워크
- adaTT 태스크 간 적응적 전이
- Uncertainty weighting (Kendall et al.)
- 로짓 전이 3가지 방식

인프라 및 실험

- 24개 ablation 시나리오
- SageMaker spot 인스턴스 활용
- Phase 0 (CPU) + Phase 1~2 (GPU) 분리
- Config-driven 파이프라인 아키텍처

문서화

프로젝트를 통해 생산된 기술 문서는 총 9편이다. 아키텍처 개요, 파이프라인 가이드, 전문가 상세, 피처 참조, PLE+adaTT 참조, Causal OT 참조, 증류 참조, 시간 참조, 규제 프레임워크가 모두 Typst 기반으로 작성되었다.

논문

연구 결과를 정리하여 arXiv에 제출할 논문 2편을 준비했다. 제한된 자원 환경에서의 대규모 멀티태스크 추천 시스템 구축 경험과, AI 에이전트를 활용한 소규모 팀의 개발 방법론을 다룬다. 국내 금융기관 실무자가 arXiv에 논문을 게재하는 최초의 사례가 될 가능성이 있다.

Ablation 분석에서 드러난 전문가 특화

24개 ablation 시나리오 분석 결과, 태스크 유형별로 전문가 특화가 명확히 드러났다. LightGCN은 multiclass 태스크(다음 상품 예측)에서, Causal 전문가는 regression 태스크(고객 가치 추정)에서 가장 큰 기여를 보였다. 이는 이종 전문가 설계의 유효성을 실증적으로 확인한 결과다.

온프렘 운영 성과

온프렘 시스템은 규제 준수 관점에서 금감원 AI RMF 24개 항목 중 85% 준수(11개 완전 + 9개 부분)를 달성했으며, 미달 항목은 기술적 구현이 아닌 조직적 의사결정 사항이었다. 12개 규제 준수 모듈(AI 고지, 거부권, 인적 재처리, 공정성 모니터링, 이해충돌 방지, 쏠림 탐지, 프롬프트 인젝션 방어, 안전성 문서, 모델 카드, 감사 추적, 동의 관리, 품질 모니터링)이 구현되었다.

평가 지표 체계

태스크 유형별 gold standard 지표를 정립했다: Binary 분류는 AUC, Multiclass 분류는 Macro F1, Regression은 MAE를 기준으로 삼았다. 단일 지표로 모든 태스크를 비교하는 오류를 방지하고, 각 태스크의 특성에 맞는 엄밀한 평가를 수행했다.

운영/감사 에이전트 — "AI가 분석하고, 사람이 판단한다"

이 프로젝트에서 가장 야심찬 설계 도전은 추천사유 생성과 별개로, 파이프라인 전체를 자율 진단하는 에이전트 시스템을 구축하는 것이었다. 핵심 질문: "전담 MLOps 인력 없이 소규모 팀이 규제를 준수하면서 AI 시스템을 운영할 수 있는가?"

설계 과정: 대화에서 아키텍처로

설계는 단일 세션의 연속 대화에서 점진적으로 발전했다:

1. **"에이전트를 넣자"** → 운영(Ops)과 감사(Audit) 분리 → 비동기 분리 근거(레이턴시, 규제 독립성, 장애 격리)
2. **"모델 수준은?"** → "대부분 룰 엔진이면 된다" → 결정론적 엔진 + LLM 대화 인터페이스 분리
3. **"온프렘은?"** → "Bedrock 없이도 기본 기능 완결" → 온프렘 baseline + AWS Bedrock 확장
4. **"할루시네이션 리스크"** → 3-에이전트 독립 투표 → **"델파이는 수렴 편향"** → 2-Round 하이브리드 (독립 투표 + 순차 심의) + 마이너리티 리포트 보존
5. **"케이스가 쌓이면 지식"** → 진단 케이스 스토어(LanceDB) → 유사 검색 + 통계 + 대응 효과 추적
6. **"한국어 모델은?"** → Solar Pro(AWS) + Exaone 3.5(온프렘) → 태스크별 최적 모델 분리

매 단계마다 "이 설계의 약점은 뭔가?"를 반복하여 구멍을 메운 결과, 15개 섹션 3,800줄의 설계 문서와 21개 파일 ~4,800줄의 구현이 하나의 세션에서 완성되었다.

핵심 설계 결정

결정론적 엔진 위에 LLM을 얹는다

에이전트의 95%는 Python 룰 엔진(if-else, 임계값, 패턴 매칭)으로 동작한다. 48개 체크리스트 항목을 자동 판정하고, finding + likely_cause + suggested_action 형식으로 리포트를 생성한다. LLM은 "이 수치를 어떻게 해석할 것인가"를 담당자와 논의하는 데만 사용한다.

이 분리의 핵심 이유: LLM의 확률적 특성은 감사 에이전트에서 오히려 리스크가 된다 — 같은 입력에 다른 진단을 내릴 수 있다. 룰 엔진이 사실을 확정하고, LLM은 그 사실의 해석을 논의한다.

마이너리티 리포트: "놓치는 것이 오탐보다 위험하다"

3-에이전트 합의에서 소수 의견(1/3)을 구조적으로 보존한다. 처음에는 순수 델파이(순차 심의)를 고려했으나, "뒤로 갈수록 앞 의견에 끌려가서 소수 의견이 사라진다"는 문제를 발견. 최종 설계: Round 1에서 독립 투표로 마이너리티를 확정하고, Round 2에서 논거만 보강 — 마이너리티는 절대 삭제 불가.

이 이름은 영화 마이너리티 리포트에서 영감을 받았다: 3명의 프리콕 중 1명만 다른 예측을 하는 것.

피처 역매핑 파이프라인 완성

추천사유 생성의 핵심 갭을 4단계로 수정했다:

1. 영문 fallback을 한국어 기본 템플릿으로 교체
2. 누락된 12개+ 피처 prefix-to-group 매핑 추가
3. ReverseMapper를 InterpretationRegistry의 Level RM fallback으로 통합
4. `generate_l1()`에 InterpretationRegistry 3-tuple enrichment 연결

이 4단계 수정으로 L1 사유에도 IG 방향 + 태스크 맥락이 반영된 한국어 해석이 포함된다.

구현 방법: 서브에이전트 병렬 구현

설계 문서 작성 → 구현 계획 수립 → Phase별 Sonnet 서브에이전트 병렬 실행 → 메인 에이전트(Opus)가 검수하는 패턴으로 구현했다. Pre-req(4건) → Phase 0(3건) → Phase 1+2(5건) → Phase 3+4(4건) → Phase 5(2건) → 미구현 보완(5건), 총 6라운드의 병렬 실행.

각 라운드마다 `py_compile` + `yaml.safe_load` + 인터페이스 계약 검증 + 하드코딩 스캔의 4단계 검수를 수행하여 "빠르게 만들고 꼼꼼히 검증"하는 리듬을 유지했다.

산출물

- **설계 문서**: 3,861줄(Typst) + 1,168줄(Markdown) + 온프렘 핸드오프 430줄
- **구현**: 21개 Python 파일, ~4,800줄
- **설정**: agent.yaml + agent_tools.yaml(38개 도구) + checklist.yaml(53개 항목)
- **문서 업데이트**: Paper 2 양쪽 + typst 10개 + design 6개 + guides 3개 = 총 20+ 파일
- **다이어그램**: Paper 2 플레이스홀더 3개 → fletcher 다이어그램, docs/typst ASCII 15개 → fletcher 변환
- **번역**: docs/typst/en tech_ref 5개 한→영 전문 번역

PaperClip 차용 (2026-04)

PaperClip (2026.3, GitHub 30K stars)의 "zero-human company" 철학은 우리 원칙과 충돌하지만, 3가지 운영 메커니즘은 차용할 가치가 있었다:

1. **Heartbeat 패턴**: 에이전트가 주기적으로 깨어나서 체크리스트를 실행하되, 변경이 없으면 `HEARTBEAT_OK`를 반환하고 다시 잠드는 구조. 우리 CP5(5분), CP6(1시간) 등 주기 점검에 적용.

2. **예산 캡 (선불 직불카드 모델)**: 에이전트별 월간 토큰 한도, 80% 소프트 경고, 100% 하드 정지. 핵심은 **graceful degradation** — 예산 초과 시 LLM 호출만 차단되고 룰 엔진은 계속 동작한다. 이것이 우리의 "온프렘 baseline이 LLM 없이 완결" 설계와 자연스럽게 연결된다.

3. **Full Tool Trace**: 모든 `ToolRegistry.call()` 호출을 자동 기록하여 "이 진단이 어떤 도구 호출을 거쳐 생성되었는가"를 완전 재현 가능하게 만든다.

LangMem의 **프롬프트 자기개선**은 의도적으로 차용하지 않았다 — 감사 관점에서 "누가 이 프롬프트를 승인했는가?"에 답할 수 없기 때문이다.

메모리 프레임워크 차용 (2026-04, 후속)

PaperClip 구현 직후, 또 다른 메모리 프레임워크들(Mem0, Zep/Graphiti, Letta, SuperLocalMemory)을 검토하였다. 여기서도 **프레임워크를 통째로 도입하지 않고** 4가지 패턴만 선택적으로 차용하였다.

Zep/Graphiti의 시간적 지식 그래프

"2026-03-15 시점에 고객 A의 상태는?" — 이 감사 질의에 답하려면 그 시점의 모델 버전, 피처, 판정이 모두 필요하다. 분산된 컴포넌트를 조인하기보다, `(엔티티, 속성, 값, valid_from, valid_to)` 스키마의 `TemporalFactStore`를 만들어 단일 필터로 해결했다. 기존 `DiagnosticCaseStore`와 같은 LanceDB 인스턴스를 공유.

SuperLocalMemory의 수학적 Decay

"3년 전 drift 해결 방식이 지금도 유효한가?" — 아니다. 최근 케이스가 더 관련성 높다. 하지만 오래된 케이스를 **삭제**하면 감사 요건(7년 보존) 위반이다. 해법: 원본은 보존하되 **검색 가중치만** $\exp\left(-\frac{\text{age}}{\tau}\right)$로 조정. 반감기 90일 기본값. `DiagnosticCaseStore.search_similar()`에 30줄 추가로 해결.

Mem0의 팩트 압축

`InterpretationRegistry`가 피처 레벨 한국어 해석을 제공하지만, 고객 **서술적 프로파일**은 없었다. "이 고객은 적금 선호, 리스크 회피"라는 팩트가 L2a 프롬프트에 없으면 Solar Pro가 맥락 부족 상태로 사유를 쓴다. `FactExtractor`를 룰 기반으로 구현 — YAML config에 15개 규칙을 정의, Python `eval()` 샌드박스(`__builtins__` 차단)로 안전하게 평가. **LLM 호출 0회**.

Letta의 Recall Memory

`BedrockDialogSession`이 세션 종료 시 대화 이력을 잃어버리던 문제를 DynamoDB 기반 `DialogRecallMemory`로 해결. 과거 대화를 임베딩 검색(또는 키워드 fallback)으로 조회하여 시스템 프롬프트에 주입한다.

품질 검수에서 발견된 연결 버그

구현 완료 후 인터페이스 계약 검수에서 **치명적 버그 1건**을 발견했다: `generate_l1()`에서 `customer_facts`를 retrieval 했지만 local 변수에만 붙였고, SQS를 통한 L2a 경로에서는 전혀 전달되지 않고 있었다. 즉 M-3 체인이 **전 구간 no-op**이었다. `get_best_reason()`에서 facts를 재조회하여 SQS context에 직접 주입하는 방식으로 수정.

이 경험이 주는 교훈: **서브에이전트가 개별 파일을 잘 구현했더라도, 파일 간 데이터 흐름을 연결하는 것은 별개의 작업**이다. 메인 에이전트의 최종 검수가 필수적인 이유.

데이터 무결성 감사: v3에서 v4로 (2026-04-10/11)

> "AUC 0.98이 나오면 처음엔 흥분한다. 그런데 모델이 버킷 함수를 역산하고 있었다는 걸 알게 되는 순간, 그 0.98은 아무 의미가 없어진다."

결정론적 리키지 발견: 18개 → 14개 태스크

4개 태스크를 제거하게 된 계기는 모델이 실제로 무엇을 학습하고 있는지 추적한 결과였다. `income_tier`는 `income`의 직접 버킷이었고, `tenure_stage`는 `tenure_months`의 버킷, `spend_level`은 `synth_monthly_spend`의 버킷이었다. `engagement_score`는 $0.3 \cdot \text{is_active} + 0.4 \cdot \text{freq} + 0.3 \cdot \text{num_products}$라는 선형 결합 그 자체였다. 네 가지 모두 모델 입력 피처 또는 그 자명한 변환이었다.

이 발견의 계기는 비정상적으로 높은 수치였다: `income_tier` AUC = 0.98. 처음에는 좋은 결과처럼 보였다. 그러나 되짚어보면 경고 신호였다 — 모델은 아무것도 학습할 필요가 없었고, 방금 입력으로 받은 버킷 경계값을 역산하기만 하면 됐다. CLAUDE.md에는 이미 "입력 피처의 결정론적 변환으로 파생되는 레이블은 태스크로 사용하지 않는다"는 원칙이 적혀 있었다. 그런데 우리 자신의 태스크 목록에는 적용하지 않았다. 각 레이블의 파생 경로를 감사한 후 4개를 제거했고, 태스크 수는 18개에서 14개로 줄었다. 남은 14개 태스크는 진정으로 불확실한 예측을 요구한다: 상품 가입, 이탈, 다음 MCC, 지출 이동 등 — 모델이 고객 행동에 대해 실질적인 무언가를 학습해야 하는 태스크들이다.

더 넓은 교훈: 리키지 검토는 **레이블 정의**를 대상으로 해야 한다, 피처 파이프라인만이 아니라. 완벽하게 깨끗한 피처 파이프라인도 그 피처들의 함수인 레이블이 있으면 무너진다.

합성 데이터 반복: v2 → v3 → v4

합성 벤치마크에서 의미 있는 레이블 분포를 얻기까지 세 번의 반복이 필요했으며, 그 과정에서 우리가 검토하지 않았던 가정들이 드러났다.

v2는 균일 난수 MCC 할당과 고정 거래 금액을 사용했다. 결과는 MCC 의존 태스크에서 거의 무작위에 가까운 레이블이었다 — 데이터 생성기가 구조 없이 만든 결과를 모델이 예측하는 상황이었다. v3에서는 페르소나 가중 MCC(4 5× 선호 배율)와 30% 거래 고착성을 도입했다. 레이블은 개선됐지만 가입 태스크가 여전히 균일에 가까웠는데, 페르소나-상품 매핑이 너무 약해서 고객을 충분히 차별화하지 못했기 때문이었다.

v4에서는 세 가지 조정을 더 강하게 가져갔다: MCC 선호 배율을 8 12×으로 높여 페르소나별 지출 패턴을 뚜렷하게 만들었고, 고착성을 60%로 올렸으며, 전 상품에 걸쳐 가입률을 높이고, 행동 변화 허용 창(mode-shift window)을 넓혔다. 결과적으로 14개 태스크 전체에서 의미 있는 분포를 얻었다 — 학습할 무언가가 있을 만큼의 분산이 있으면서, 자명한 예측기가 지배하지 않는 수준이었다.

세 번의 반복에서 나타난 패턴: 각 수정이 **다음** 가정의 약점을 드러냈다. v2에서 MCC 무작위성을 수정하자 페르소나 매핑 문제가 보였다. 페르소나 매핑을 수정하자 가입률 문제가 보였다. 합성 데이터 설계는 본질적으로 반복적이며, 반복이 끝났다는 올바른 신호는 생성기 코드가 깔끔해 보일 때가 아니라 레이블 분포가 목표 도메인을 닮았을 때다.

HGCN vs. LightGCN 역할 혼동

두 그래프 전문가는 처음에 같은 종류의 데이터로 라우팅되어 있었다: 상품 공동 보유 관계. HGCN이 `product_hierarchy`(상품-고객 이분 그래프)를 받아서 LightGCN과 기능적으로 동일해졌다. 두 전문가 모두 협업 필터링 방식의 친화성을 학습했다. 온프렘 설계 의도 — HGCN은 **쌍곡 공간에서 트리 구조**를 학습하고 LightGCN은 **이분 친화성**을 학습한다 — 가 라우팅에 반영되지 않았던 것이다.

수정을 위해 `merchant_hierarchy` 생성기를 다시 작성하여 실제 MCC L1→L2 푸앵카레 임베딩(27차원)을 생성하도록 했다: 상인 카테고리 그룹과 하위 카테고리 간의 계층적 관계를 표현한다. `feature_groups.yaml`을 갱신하여 HGCN의 `target_experts`가 상인 계층 그룹을 가리키도록 하고, LightGCN은 상품 공동 보유를 유지하도록 했다. 이 변경 이후 두 전문가는 진정으로 다른 기능을 수행한다: HGCN은 쌍곡 공간에서 분류 거리를 인코딩하고, LightGCN은 구매 공동 발생 패턴을 협업 필터링으로 인코딩한다.

이것은 코드 버그가 아니라 설계 의도 보존 문제였다. 생성기는 유효한 임베딩을 만들었고, 라우팅이 잘못된 전문가에게 보냈다. Config 기반 라우팅(`feature_groups.yaml`)만이 이 문제를 지속 가능하게 관리할 수 있다 — 어댑터나 모델 코드에 하드코딩된 라우팅이었다면 잘못된 할당이 훨씬 더 오래 보이지 않았을 것이다.

인프라 및 도구 수정

이 세션에서 여러 인프라 문제가 발견되어 수정되었다.

전문가 라우팅 단위. 라우팅은 컬럼 수준이 아니라 피처 그룹 수준으로 지정해야 한다. 컬럼 수준 라우팅은 Phase 0 정규화 재정렬에서 살아남지 못한다: 로그 접미사 컬럼이 추가되면 컬럼 인덱스가 이동하고 라우팅이 조용히 깨진다. 피처 그룹 수준 라우팅은 `feature_groups.yaml`의 명명된 그룹을 사용하며 정규화 변환 전반에서 안정적이다.

피처 그룹 범위 인덱싱. `feature_group_ranges`는 그룹 내 전체 컬럼의 최솟값/최댓값 위치로 연속 블록을 구성해야 한다. 이전 로직은 `_log` 접미사 컬럼이 기본 컬럼 뒤에 추가될 때 깨졌다: 범위 끝점이 더 이상 올바른 인덱스를 포함하지 않았다.

태스크 유형별 지표 집계. 태스크 유형에 관계없이 AUC를 평균 내는 것은 오해를 낳는다. 올바른 집계는 이진 분류 태스크에 평균 AUC, 다중 클래스에 평균 F1-macro, 회귀에 평균 MAE를 사용한다. 유형 혼합 평균화는 개별 태스크 유형에 대한 모델 동작을 가리는 지표를 만들어냈다.

Ablation 결과 보존. 결과는 보관해야 하며 삭제해서는 안 된다. 정리 스크립트의 `rm -rf` 패턴을 아카이브 순환으로 대체했다: 새 결과를 쓰기 전에 각 실행의 결과가 타임스탬프가 붙은 하위 디렉터리로 이동된다. 이로써 완료된 ablation 실행의 우발적 손실을 방지한다.

Windows 슬립 및 서브프로세스 안정성. Windows의 절전 모드가 야간 실행을 중단시킨다. 장시간 세션을 위해 유휴 시 절전을 비활성화했다. 정상 실행에서 0이 아닌 종료 코드가 반환되는 산발적 서브프로세스 오류는 1초 백오프의 재시도 로직으로 해결했다.

LGBM 학생 모델의 온도 스케일링. 지식 증류 논문은 일반적으로 신경망 학생 모델에 T = 3 5를 권장한다. 트리 기반 LGBM 학생에는 T = 1이 적절함을 확인했다: LGBM은 높은 온도 소프트닝이 생성하는 부드러운 확률 분포를 표현할 수 없으므로, 신경망 학생 온도 설정은 오히려 해로웠다. T = 1은 교사의 순위 순서를 보존하면서 학생에게 불가능한 분포 형태를 강요하지 않는다.

향후 계획

학술 및 업계 발표

- **arXiv 논문 게재**: 2편 (시스템 아키텍처 논문 + AI 에이전트 개발 방법론 논문)
- **Anthropic 케이스 스터디 제출**: Claude Code를 활용한 금융 AI 시스템 구축 사례
- **GARP 제출**: FRM 자격과 AI 리스크 관리를 결합한 관점의 논문

규제 및 제도 대응

- **금감원 AI 기본법 컴플라이언스 검토 요청**: AI 기본법 시행령 및 가이드라인이 수립되는 시점에 맞추어, 본 시스템의 설명 가능성 프레임워크에 대한 검토를 요청할 계획이다.

후속 작업

- **온프렘 운영 데이터 결과**: 실제 운영 데이터에서의 성능 결과를 논문 보충 자료로 추가
- **공개 GitHub 저장소**: 조직 정보를 제거한 sanitized 버전의 코드를 공개 저장소로 공개

이 프로젝트는 “자원의 부족”이 아니라 “자원의 재정의”를 통해 완성되었다.
데스크톱 GPU 1대와 AI 에이전트들의 조합이 전용 인프라를 대체할 수 있음을 보여준 사례다.